

Assignment S2-6: Write a YACC Program to Generate Pseudo Assembly Code for Assignment Statements and Arithmetic Expressions

1. Objective

The objective of this laboratory is to develop a **YACC** program that generates **Pseudo Assembly Code (PAC)** for assignment statements and arithmetic expressions using **Syntax-Directed Definitions (SDDs)**.

2. Learning Outcomes

After completing this laboratory, students will be able to:

- write a YACC grammar for assignment statements and arithmetic expressions;
- use **Syntax-Directed Definitions (SDDs)** to specify semantic rules;
- implement semantic actions in YACC;
- generate temporary variables for intermediate results; and
- translate arithmetic expressions into pseudo assembly code.

3. Background

A compiler translates a high-level program into one or more lower-level representations. In this laboratory, the compiler translates source statements directly into **Pseudo Assembly Code (PAC)**.

Source Program



Lexical Analysis (Lex)



Syntax Analysis (YACC)



Pseudo Assembly Code (PAC)

The translation is performed using **Syntax-Directed Definitions (SDDs)**. An **SDD** consists of:

- a context-free grammar,
- attributes associated with grammar symbols, and
- semantic rules attached to grammar productions.

Whenever the parser recognizes a production, the corresponding semantic rules are executed to generate pseudo assembly instructions.

In **YACC**, these semantic rules are implemented using **semantic actions**.

4. Problem Statement

Develop a **Lex/YACC** program that accepts assignment statements containing arithmetic expressions and generates equivalent **Pseudo Assembly Code (PAC)**.

The generated code should preserve operator precedence and associativity.

5. Language Specification

Your compiler should support

Assignment Statement

id = expression ;

Example: a = b + c;

Assignment S2-6: Write a YACC Program to Generate Pseudo Assembly Code for Assignment Statements and Arithmetic Expressions

Arithmetic Operators

+
-
*
/
%

Operands

- identifiers
- integer constants

Parentheses

Expressions enclosed within parentheses must be supported.

Example: a = (b+c)*d;

6. Grammar

The following grammar may be used.

Program → StmtList

StmtList → StmtList Stmt
 | Stmt

Stmt → id '=' Expr ';' ;

Expr → Expr '+' Term
 | Expr '-' Term
 | Term

Term → Term '*' Factor
 | Term '/' Factor
 | Term '%' Factor
 | Factor

Factor → '(' Expr ')'
 | id
 | number

Students may modify the grammar if necessary.

7. Syntax-Directed Definition

Each expression should maintain a synthesized attribute named **place** that stores the location of the computed result.

Example semantic rules

Production

Expr → Expr1 + Term

Assignment S2-6: Write a YACC Program to Generate Pseudo Assembly Code for Assignment Statements and Arithmetic Expressions

Semantic Rule

Expr.place = newTemp()

```
emit(  
    ADD Expr.place,  
        Expr1.place,  
        Term.place  
)
```

Production

Stmt $\rightarrow$ id = Expr ;

Semantic Rule

```
emit(  
    MOV id,  
        Expr.place  
)
```

These semantic rules should be implemented as **semantic actions** in the YACC program.

8. Temporary Variables

Intermediate results should be stored in automatically generated temporary variables.

Example

T1

T2

T3

...

A helper function such as- *newTemp()* may be used to generate temporary names.

9. Sample Input

a = b + c;

x = (a+b)*c;

y = a*b + c*d;

10. Sample Output

ADD T1, b, c

MOV a, T1

ADD T2, a, b

MUL T3, T2, c

MOV x, T3

MUL T4, a, b

MUL T5, c, d

ADD T6, T4, T5

MOV y, T6

Assignment S2-6: Write a YACC Program to Generate Pseudo Assembly Code for Assignment Statements and Arithmetic Expressions

11. Student Tasks

Implement the following components.

1. Lexical analyzer using **Lex/Flex**.
2. Parser using **YACC/Bison**.
3. Grammar for assignment statements and arithmetic expressions.
4. Semantic actions based on the given SDD.
5. Temporary variable generator.
6. Pseudo assembly code generator.
7. Syntax error reporting.

12. Submission Requirements

Students should submit

- Lex source file (.l)
- YACC source file (.y)
- Source code
- Sample input and output
- A brief report (2–3 pages) describing:
 - the grammar,
 - the SDD,
 - the semantic actions, and
 - the generated pseudo assembly code.

Assignment S2-6: Write a YACC Program to Generate Pseudo Assembly Code for Assignment Statements and Arithmetic Expressions

Appendix A: Formal Definition of Pseudo Assembly Code (PAC)

Definition

Pseudo Assembly Code (PAC) is a simplified, machine-independent assembly language used as an educational intermediate representation. It demonstrates how high-level language constructs are translated into low-level instructions during compilation.

Temporary Variables

Intermediate results are stored in temporary variables.

T1, T2, T3, ...

Instruction Set

Data Movement

MOV destination, source

Arithmetic Instructions

ADD destination, operand1, operand2

SUB destination, operand1, operand2

MUL destination, operand1, operand2

DIV destination, operand1, operand2

MOD destination, operand1, operand2

Programming Rules

1. Every arithmetic operation generates a new temporary variable.
2. The final expression result is assigned to the destination variable using MOV.
3. Operator precedence and associativity must be preserved.
4. Temporary variables should be generated sequentially (T1, T2, T3, ...).

Example

Source Statement:

$a = (b + c) * d;$

Pseudo Assembly Code:

ADD T1, b, c

MUL T2, T1, d

MOV a, T2

This manual keeps the focus on the essential concepts—**SDDs, YACC semantic actions, and pseudo assembly generation**—while avoiding implementation details that students are expected to design themselves. It also serves as a solid foundation for subsequent laboratory assignments on relational expressions, branching statements, and loops.